```
import numpy as np
import sympy as sp
import matplotlib.pyplot as plt
```

```
#Usando método da bissecção
equacao = lambda x:x*np.log(x)

ini_intervalo = 2
fim_intervalo = 4
erro = 10**-3
max_iteracao = 6
ponto_med = lambda ini_intervalo,fim_intervalo: (ini_intervalo+fim_intervalo)/2
for i in range(max_iteracao):

  new_x = ponto_med(ini_intervalo,fim_intervalo)
  erro_at = abs(ini_intervalo-fim_intervalo)/ max(1,abs(fim_intervalo))
  if erro_at < erro:
    print(f"A raiz está em {new_x}")
    break
  elif equacao(ini_intervalo)*equacao(new_x) < 0:
    fim_intervalo = new_x
  else:
    ini_intervalo = new_x
  #só exec o else se o loop for concluido, o else pd ser anexado a um loop
else:
  print("Não chegou no erro após",max_iteracao,"iterações. Última aproximação foi",new_x)
```

```
Não chegou no erro após 6 iterações. Última aproximação foi 3.96875
```

```
ponto_med = lambda ini_intervalo,fim_intervalo: (ini_intervalo+fim_intervalo)/2
def bisseccao_(ini_intervalo,fim_intervalo,erro,max_iteracao,equacao):
  for i in range(max_iteracao):

    new_x = ponto_med(ini_intervalo,fim_intervalo)
    erro_at = abs(ini_intervalo-fim_intervalo)/ max(1,abs(fim_intervalo))
    if erro_at < erro:
      print(f"A raiz está em {new_x}")
      return new_x,i+1
    elif equacao(ini_intervalo)*equacao(new_x) < 0:
      fim_intervalo = new_x
    else:
      ini_intervalo = new_x
  #só exec o else se o loop for concluido, o else pd ser anexado a um loop
  else:
    print("Não chegou no erro após",max_iteracao,"iterações. Última aproximação foi",new_x)
    return new_x, max_iteracao # Retorna a última aproximação e o número máximo de iterações

bisseccao_(0.5,1.5,10**-3,100,lambda x:x*np.log(x))
```

```
A raiz está em 1.49951171875
(1.49951171875, 11)
```

```
#Meu da posi_falsa
def posi_falsa(ini_intervalo,fim_intervalo,erro,max_iteracao,equacao):
  for i in range(max_iteracao):
    denominador = equacao(fim_intervalo) - equacao(ini_intervalo)
    if abs(denominador)<1e-10:
      print("Divisão por zero ou denominador muito pequeno")
      break
```

```
    new_x = (ini_intervalo*equacao(fim_intervalo)-fim_intervalo*equacao(ini_intervalo))/deno
    erro_at = abs(equacao(new_x))
    if erro_at < erro:
      print(f"A raiz está em {new_x} após {i+1} iterações. f(raiz) = {equacao(new_x):.6f}")
      return new_x,i+1
    elif equacao(ini_intervalo)*equacao(new_x) < 0:
      fim_intervalo = new_x
    else:
      ini_intervalo = new_x
  #só exec o else se o loop for concluido, o else pd ser anexado a um loop
  else:
    print("Não chegou no erro após",max_iteracao,"iterações. Última aproximação foi",new_x)
    return new_x, max_iteracao # Adicionar retorno explícito

posi_falsa(0.5,1.5,10**-3,100,lambda x:x*np.log(x))
```

```
A raiz está em 0.9991317962450351 após 4 iterações. f(raiz) = -0.000868
(np.float64(0.9991317962450351), 4)
```

```
ponto_med = lambda ini_intervalo,fim_intervalo: (ini_intervalo+fim_intervalo)/2
def phi(x,equacao):
  return x - equacao(x)

def ponto_fixo(ini_intervalo,fim_intervalo,erro,max_iteracao,equacao):
  chute_ini = ponto_med(ini_intervalo,fim_intervalo)
  xAnt = chute_ini
  for i in range(max_iteracao):
    x_atual = phi(xAnt,equacao)
    if np.isnan(x_atual) or np.isinf(x_atual) or x_atual <= 0:
      print(f"O método divergiu na iteração {i+1}. O valor de x se tornou inválido ({x_atual}
      return x_atual, i+1
    erro_at = abs(x_atual - xAnt)/max(1,abs(x_atual))

    if erro_at < erro:
      print(f"A raiz está em {x_atual:.6f} após {i+1} iterações. f(raiz) = {equacao(x_atual):.
      return x_atual,i+1

    xAnt = x_atual
  #só exec o else se o loop for concluido, o else pd ser anexado a um loop
  else:
    print("Não chegou no erro após",max_iteracao,"iterações. Última aproximação foi",x_atual)
    return x_atual, max_iteracao # Adicionar retorno explícito

ponto_fixo(0.5,1.5,10**-3,100,lambda x:x*np.log(x))
```

```
A raiz está em 1.000000 após 1 iterações. f(raiz) = 0.000000
(np.float64(1.0), 1)
```

```
#Método de newton
ponto_med = lambda ini_intervalo,fim_intervalo: (ini_intervalo+fim_intervalo)/2
def derivada(equacao,x):
  return sp.diff(equacao,x)

def newton_(ini_intervalo,fim_intervalo,erro,max_iteracao,equacao):
  #O simpy e numpy tem leituras != então é preciso transformas os dados de leituras das funç
  x_simb = sp.Symbol('x')
  #transforma para simpy
  expressao = equacao(x_simb)
  #calc a derivada, a equação
  derivada_simb = derivada(expressao,x_simb)
  #transforma de volta a derivada para numpy, é uma função de derivada da equação fornecida
  derivada_num = sp.lambdify(x_simb,derivada_simb,'numpy')

  chute_ini = ponto_med(ini_intervalo,fim_intervalo)
```

```
  xAnt = chute_ini

  for i in range(max_iteracao):
    derivadaEq = derivada_num(xAnt)
    if abs(derivadaEq) < 1e-10:
      print("Divisão por zero ou denominador muito pequeno")
      return xAnt, i+1 # Retorna o último valor conhecido e a iteração atual em caso de divi

    x_atual = xAnt - equacao(xAnt)/derivadaEq
    x_atual = float(x_atual) # Converte x_atual para float para garantir compatibilidade nas

    erro_at = abs(x_atual - xAnt)/max(1,abs(x_atual))

    if erro_at < erro:
      print(f"A raiz está em {x_atual:.6f} após {i+1} iterações. f(raiz) = {equacao(x_atual)
      return x_atual,i+1

    xAnt = x_atual
  #só exec o else se o loop for concluido, o else pd ser anexado a um loop
  else:
    print("Não chegou no erro após",max_iteracao,"iterações. Última aproximação foi",x_atual
    return x_atual, max_iteracao # Adicionar retorno explícito

newton_(0.5,1.5,10**-3,100,lambda x:x*sp.log(x))
```

```
A raiz está em 1.000000 após 1 iterações. f(raiz) = 0.000000
(1.0, 1)
```

```
#Método da Secante
ponto_med = lambda ini_intervalo,fim_intervalo: (ini_intervalo+fim_intervalo)/2
def secante(ini_intervalo,fim_intervalo,erro,max_iteracao,equacao):
  chute1 = ini_intervalo
  chute2 = fim_intervalo
  for i in range(max_iteracao):

    denominador = (equacao(chute2)-equacao(chute1))
    if abs(denominador) < 1e-10:
      print("Divisão por zero ou denominador muito pequeno")
      return float(chute2), i+1 # Retorna o último valor conhecido e a iteração atual em cas
    new_x = chute2 - equacao(chute2)*(chute2-chute1)/denominador
    erro_at = abs(new_x - chute2)/max(1,abs(new_x))
    if erro_at < erro:
      print(f"A raiz está em {new_x} após {i+1} iterações. f(raiz) = {equacao(new_x):.6f}")
      return float(new_x),i+1 # Adicionar o número de iterações ao retorno
    else:
      chute1,chute2 = chute2,new_x
  else:
    print("Não chegou no erro após",max_iteracao,"iterações. Última aproximação foi",new_x)
    return float(new_x), max_iteracao # Adicionar retorno explícito

secante(2,4,10**-3,1000,lambda x:x*np.log(x))
```

```
A raiz está em 1.000007736443409 após 5 iterações. f(raiz) = 0.000008
(1.000007736443409, 5)
```

```
#bissecção sem max de iteração
ponto_med = lambda ini_intervalo,fim_intervalo: (ini_intervalo+fim_intervalo)/2
def bisseccao_si(ini_intervalo,fim_intervalo,erro,equacao):
  while True:

    erro_at = abs(ini_intervalo-fim_intervalo)/ max(1,abs(fim_intervalo))
    new_x = ponto_med(ini_intervalo,fim_intervalo)
    if erro_at < erro:
      print(f"A raiz está em {new_x}")
```

```
      return new_x,i+1
    elif equacao(ini_intervalo)*equacao(new_x) < 0:
      fim_intervalo = new_x
    else:
      ini_intervalo = new_x

bisseccao_si(0.5,1.5,10**-3,lambda x:x*np.log(x))
bisseccao_si(0,1,10**-3,lambda x:x*np.euler_gamma**-x -np.euler_gamma**-3)
```

```
A raiz está em 1.49951171875
A raiz está em 0.99951171875
(0.99951171875, 6)
```

```
#Posição falsa sem máximo de iterações
def posi_falsa_si(ini_intervalo,fim_intervalo,erro,equacao):
  if(equacao(ini_intervalo)*equacao(fim_intervalo)>0):
    print("\nA raiz não esta necessariamente no intervalo")
  i=0
  while True:
    denominador = equacao(fim_intervalo) - equacao(ini_intervalo)
    if abs(denominador)<1e-10:
      print("Divisão por zero ou denominador muito pequeno")
      break
    new_x = (ini_intervalo*equacao(fim_intervalo)-fim_intervalo*equacao(ini_intervalo))/deno
    erro_at = abs(equacao(new_x))
    if erro_at < erro:
      print(f"A raiz está em {new_x} após {i+1} iterações. f(raiz) = {equacao(new_x):.6f}")
      return new_x,i+1
    elif equacao(ini_intervalo)*equacao(new_x) < 0:
      fim_intervalo = new_x
    else:
      ini_intervalo = new_x
    i+=1

posi_falsa_si(0.5,1.5,10**-3,lambda x:x*np.log(x))
posi_falsa_si(0,1,10**-3,lambda x:x*np.euler_gamma**-x -np.euler_gamma**-3)
```

```
A raiz está em 0.9991317962450351 após 4 iterações. f(raiz) = -0.000868

A raiz não esta necessariamente no intervalo
A raiz está em 1.865357220902013 após 12 iterações. f(raiz) = -0.000407
(1.865357220902013, 12)
```

```
#Ponto fixo sem max de iteração
ponto_med = lambda ini_intervalo,fim_intervalo: (ini_intervalo+fim_intervalo)/2
def phi(x,equacao):
  return x - equacao(x)

def ponto_fixo_si(ini_intervalo,fim_intervalo,erro,equacao):
  chute_ini = ponto_med(ini_intervalo,fim_intervalo)
  xAnt = chute_ini
  i=0
  while True:
    x_atual = phi(xAnt,equacao)

    erro_at = abs(x_atual - xAnt)/max(1,abs(x_atual))

    if erro_at < erro:
      print(f"A raiz está em {x_atual:.6f} após {i+1} iterações. f(raiz) = {equacao(x_atual)
      return x_atual,i+1

    xAnt = x_atual
    i+=1
```

```
ponto_fixo_si(0.5,1.5,10**-3,lambda x:x*np.log(x))
ponto_fixo_si(0,1,10**-3,lambda x:x*np.euler_gamma**-x -np.euler_gamma**-3)
```

```
A raiz está em 1.000000 após 1 iterações. f(raiz) = 0.000000
A raiz está em 1.866309 após 38769 iterações. f(raiz) = 0.004970
(1.8663094551724715, 38769)
```

```
#Secante sem máximo de iterações
ponto_med = lambda ini_intervalo,fim_intervalo: (ini_intervalo+fim_intervalo)/2
def secante_si(ini_intervalo,fim_intervalo,erro,equacao):
  chute1 = ini_intervalo
  chute2 = fim_intervalo
  i=0
  while True:

    denominador = (equacao(chute2)-equacao(chute1))
    if abs(denominador) < 1e-10:
      print("Divisão por zero ou denominador muito pequeno")
      break
    new_x = chute2 - equacao(chute2)*(chute2-chute1)/denominador
    erro_at = abs(new_x - chute2)/max(1,abs(new_x))
    if erro_at < erro:
      print(f"A raiz está em {new_x} após {i+1} iterações. f(raiz) = {equacao(new_x):.6f}")
      return float(new_x),i+1
    else:
      chute1,chute2 = chute2,new_x
      i+=1

secante_si(2,4,10**-3,lambda x:x*np.log(x))
secante_si(0,1,10**-3,lambda x:x*np.euler_gamma**-x -np.euler_gamma**-3)
```

```
A raiz está em 1.000007736443409 após 5 iterações. f(raiz) = 0.000008
A raiz está em 1.865414826356055 após 6 iterações. f(raiz) = -0.000082
(1.865414826356055, 6)
```

```
#Método de newton sem máximo de iterações
ponto_med = lambda ini_intervalo,fim_intervalo: (ini_intervalo+fim_intervalo)/2
def derivada(equacao,x):
  return sp.diff(equacao,x)

def newton_si(ini_intervalo,fim_intervalo,erro,equacao):
  #O simpy e numpy tem leituras != então é preciso transformas os dados de leituras das funç
  x_simb = sp.Symbol('x')
  #transforma para simpy
  expressao = equacao(x_simb)
  #calc a derivada, a equação
  derivada_simb = derivada(expressao,x_simb)
  #transforma de volta a derivada para numpy, é uma função de derivada da equação fornecida
  derivada_num = sp.lambdify(x_simb,derivada_simb,'numpy')

  chute_ini = ponto_med(ini_intervalo,fim_intervalo)
  xAnt = chute_ini
  i=0
  while True:
    derivadaEq = derivada_num(xAnt)
    if abs(derivadaEq) < 1e-10:
      print("Divisão por zero ou denominador muito pequeno")
      break

    x_atual = xAnt - equacao(xAnt)/derivadaEq

    erro_at = abs(x_atual - xAnt)/max(1,abs(x_atual))

    if erro_at < erro:
```

```
        print(f"A raiz está em {x_atual:.6f} após {i+1} iterações. f(raiz) = {equacao(x_atual)
        return x_atual,i+1

    xAnt = x_atual
    i+=1

newton_si(0.5,1.5,10**-3,lambda x:x*sp.log(x))
newton_si(0,1,10**-3,lambda x:x*np.euler_gamma**-x -np.euler_gamma**-3)
```

```
A raiz está em 1.000000 após 1 iterações. f(raiz) = 0.000000
A raiz está em 1.865429 após 6 iterações. f(raiz) = 0.000000
(1.8654292896281737, 6)
```

```
#Exercício
x_bi,i_bi = bisseccao_si(0,1,10**-3,lambda x:x*np.euler_gamma**-x -np.euler_gamma**-3)
x_pf,i_pf = posi_falsa_si(0,1,10**-3,lambda x:x*np.euler_gamma**-x -np.euler_gamma**-3)
x_pntF,i_pntF = ponto_fixo_si(0,1,10**-3,lambda x:x*np.euler_gamma**-x -np.euler_gamma**-3)
x_n,i_n = newton_si(0,1,10**-3,lambda x:x*np.euler_gamma**-x -np.euler_gamma**-3)
x_s,i_s = secante_si(0,1,10**-3,lambda x:x*np.euler_gamma**-x -np.euler_gamma**-3)
metodos = ["Bissecção", "Falsa Posição", "Ponto Fixo", "Newton", "Secante"]
resultados = []

# Guardando as variáveis que você chamou no código

try: resultados.append([metodos[0], f"{x_bi:.5f}", i_bi])
except: resultados.append([metodos[0], "Fora de Intervalo", "-"])

try: resultados.append([metodos[1], f"{x_pf:.5f}", i_pf])
except: resultados.append([metodos[1], "Fora de Intervalo", "-"])

try: resultados.append([metodos[2], f"{x_pntF:.5f}", i_pntF])
except: resultados.append([metodos[2], f"{x_pntF:.5f}", i_pntF])

try: resultados.append([metodos[3], f"{x_n:.5f}", i_n])
except: resultados.append([metodos[3], f"{x_n:.5f}", i_n])

try: resultados.append([metodos[4], f"{x_s:.5f}", i_s])
except: resultados.append([metodos[4], f"{x_s:.5f}", i_s])

# Plotando a tabela
fig, ax = plt.subplots(figsize=(10, 4))
ax.axis('off')

tabela = ax.table(cellText=resultados,
                  colLabels=["Método", "Raiz (x)", "Iterações (i)"],
                  loc='center',
                  cellLoc='center')
plt.title("Comparativo de Convergência (Intervalo [0, 1])", fontsize=14, pad=20)
plt.show()
```

```
A raiz está em 0.99951171875

A raiz não esta necessariamente no intervalo
A raiz está em 1.865357220902013 após 12 iterações. f(raiz) = -0.000407
A raiz está em 1.866309 após 38769 iterações. f(raiz) = 0.004970
A raiz está em 1.865429 após 6 iterações. f(raiz) = 0.000000
A raiz está em 1.865414826356055 após 6 iterações. f(raiz) = -0.000082
```

Comparativo de Convergência (Intervalo [0, 1])

Método	Raiz (x)	Iterações (i)
Bissecção	0.99951	6
Falsa Posição	1.86536	12
Ponto Fixo	1.86631	38769
Newton	1.86543	6
Secante	1.86541	6

```
#Exercício 1
x1_bi,it_bi = bisseccao_(2,4,10**-3,4,lambda x:x*np.log(x))
x1_pf,it_pf = posi_falsa(2,4,10**-3,4,lambda x:x*np.log(x))
x1_pntF,it_pntF = ponto_fixo(2,4,10**-3,4,lambda x:x*np.log(x))
x1_n,it_n = newton_(2,4,10**-3,4,lambda x:x*sp.log(x))
x1_s,it_s = secante(2,4,10**-3,4,lambda x:x*np.log(x))

metodos = ["Bissecção", "Falsa Posição", "Ponto Fixo", "Newton", "Secante"]
aproximacoes = [
    [metodos[0], f"{x1_bi:.8f}"],
    [metodos[1], f"{x1_pf:.8f}"],
    [metodos[2], f"{x1_pntF:.8f}"],
    [metodos[3], f"{x1_n:.8f}"],
    [metodos[4], f"{x1_s:.8f}"]
]

fig, ax = plt.subplots(figsize=(6, 4))
ax.axis('off')

# Criando a tabela
tabela = ax.table(
    cellText=aproximacoes,
    colLabels=["Método", "Aproximação da Raiz"],
    loc='center',
    cellLoc='center'
)
```

```
Não chegou no erro após 4 iterações. Última aproximação foi 3.875
Não chegou no erro após 4 iterações. Última aproximação foi 1.0263729718286507
O método divergiu na iteração 1. O valor de x se tornou inválido (-0.2958368660043291).
Não chegou no erro após 4 iterações. Última aproximação foi 1.0000008411696406
Não chegou no erro após 4 iterações. Última aproximação foi 1.0009706977999608
```

Método	Aproximação da Raiz
Bissecção	3.87500000
Falsa Posição	1.02637297
Ponto Fixo	-0.29583687
Newton	1.00000084
Secante	1.00097070